

点亮LED灯

点亮LED灯

- 一、学习目标
- 二、硬件搭建
- 三、实验步骤
 - 1. 打开SYSCONFIG配置工具
 - 2. 参数设置
 - 2.1 设置端口参数
 - 2.2 设置引脚参数
 - 2.3 保存更新配置文件
 - 3. 写入程序
 - 4. 编译
- 四、程序分析
- 五、实验现象

一、学习目标

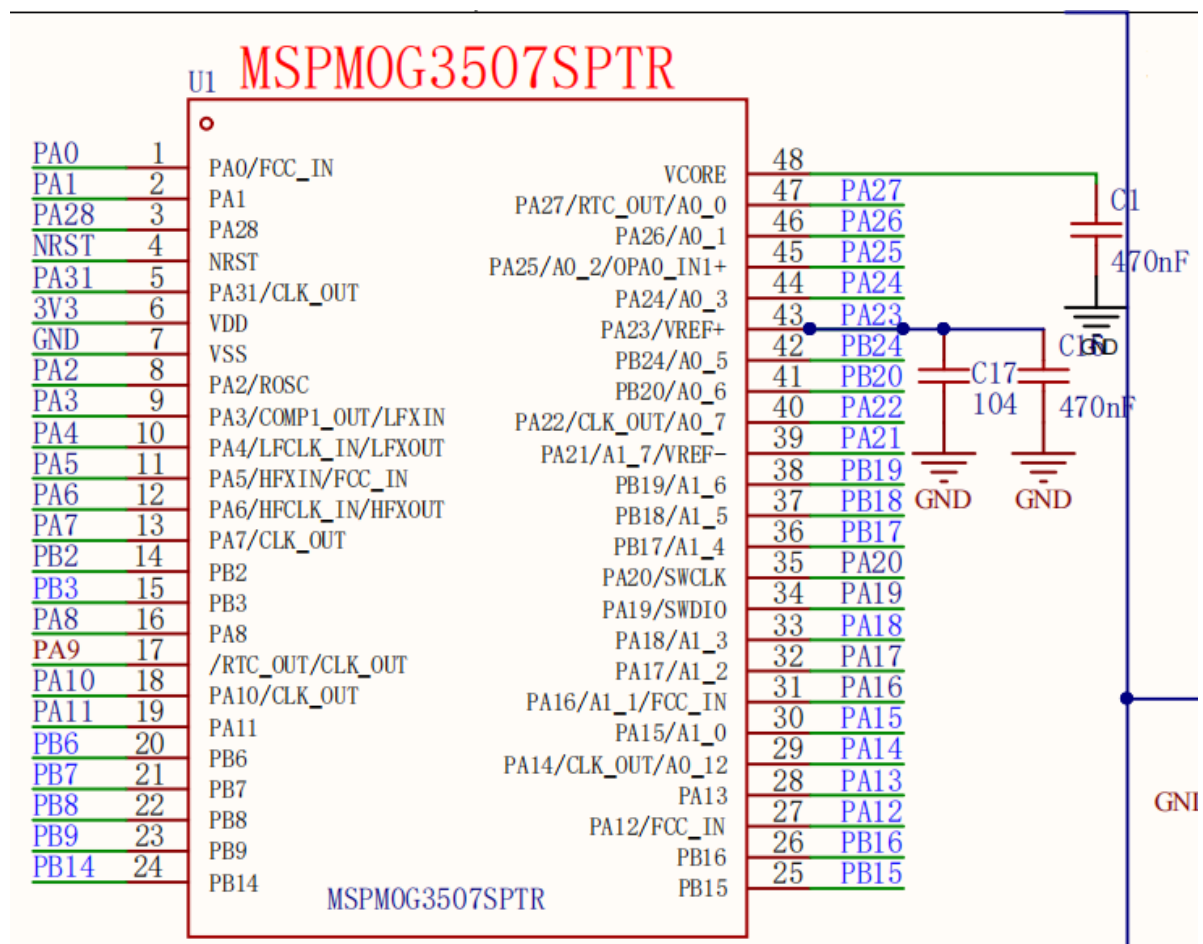
- 1.学习MSPM0G3507主板的引脚基本使用。
- 2.了解如何控制板载的LED灯。

二、硬件搭建

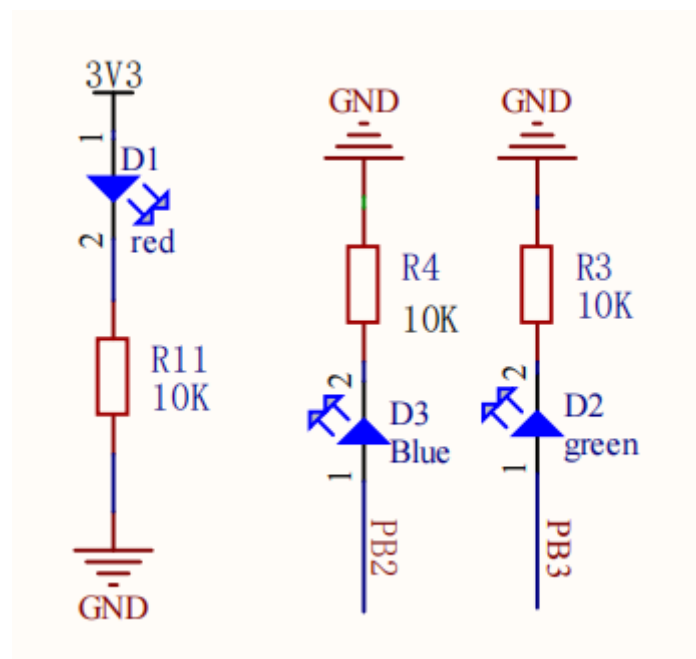
本次课程无需额外的硬件设备，直接利用MSPM0G3507主板上的板载LED灯即可。

我们在主板上设置了两个LED用户灯，用户可以自行DIY其功能。下面以**LED1**例。

MSPM0G3507主控图



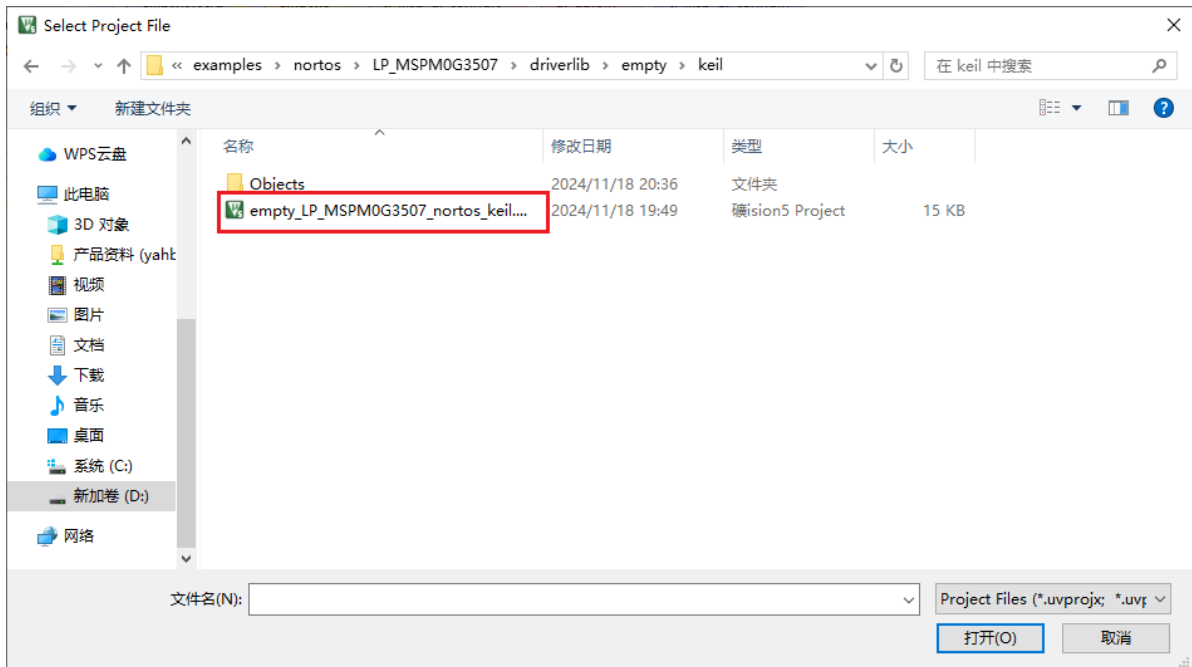
LED部分原理图



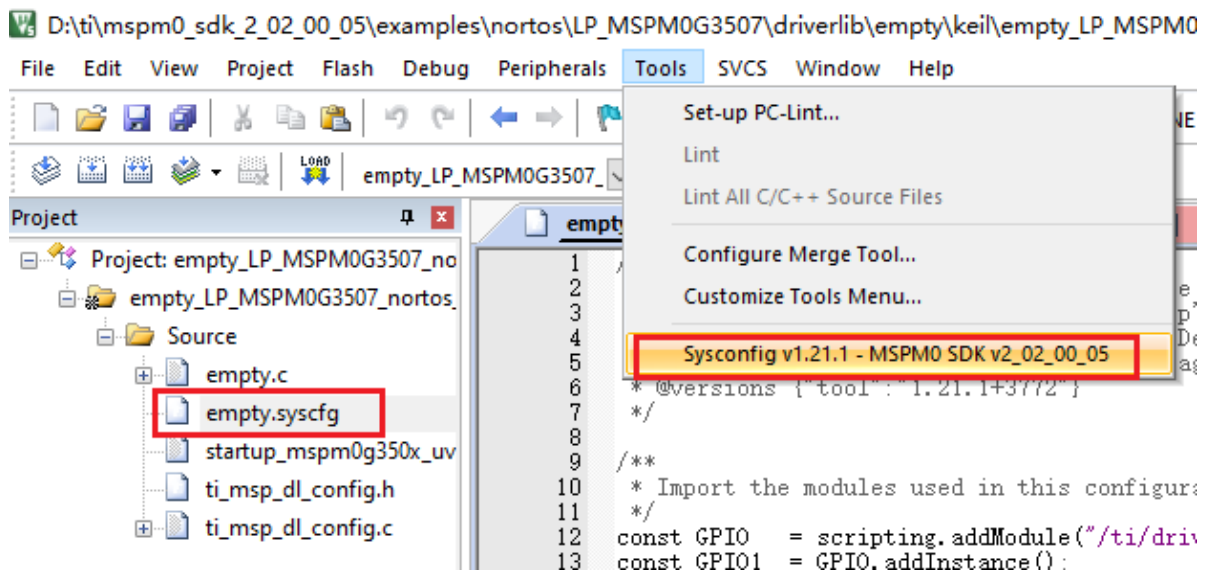
三、实验步骤

1. 打开SYSCONFIG配置工具

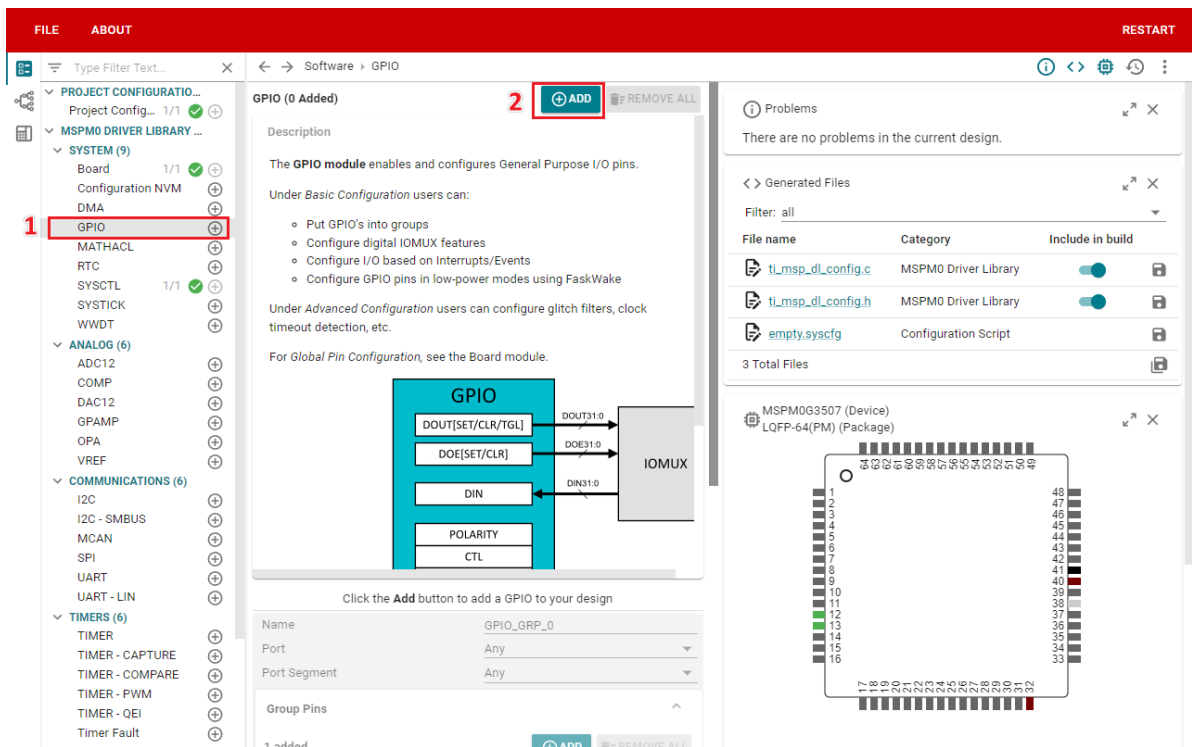
在KEIL中打开SDK中的空白工程 empty。



选中之后打开，在keil界面打开 empty.syscfg 文件，在 **empty.syscfg** 文件打开情况下，再在 Tools 栏选择打开 SYSCONFIG 的GUI界面。



在 SYSCONFIG 中，左侧选择 MCU 外设，找到 GPIO 选项并点击进入。在 GPIO 中点击 ADD，添加一组GPIO。

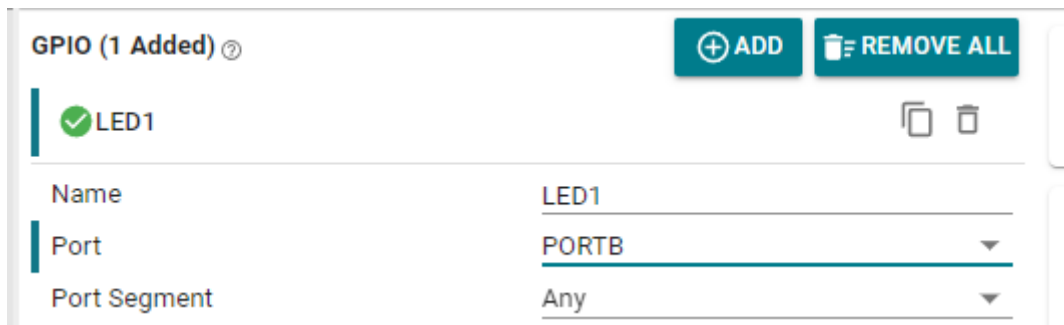


2. 参数设置

2.1 设置端口参数

设置GPIO参数，将该引脚命名为 LED1

由LED部分原理图可知，LED1所在的端口为 PB2，所以设置Port为 PORTB。



参数说明：

Name：GPIO实例的自定义名称。默认情况下，名称从数字后缀“0”开始；我们可自定义名称以反映模块用途（例如，将 GPIO 命名为“LED1”，这样我们就知道这个引脚是专门用于控制LED1）。

Port：GPIO实例所在端口。LED是接到了GPIOB2引脚，只能选择PORTB。

Port Segment：设置端口上下拉电阻。注意是端口上下拉，是设置整个GPIOB端口。

2.2 设置引脚参数

设置引脚为输出模式，设置引脚默认输出低电平，设置引脚为下拉模式，设置引脚为GPIOB_2引脚，不开启中断事件。

1 added

⊕ ADD

🗑️ REMOVE ALL

✓ PIN_2



Name	PIN_2
Direction	Output ▼
Initial Value	Cleared ▼
IO Structure	Any ▼

Digital IOMUX Features



Internal Resistor	Pull-Down Resistor ▼
Invert	Disabled ▼
Drive Strength Control	Low ▼
High-Impedance	Disabled ▼

Assigned Port	PORTB ▼
Assigned Port Segment	Any ▼
Assigned Pin	2

Interrupts/Events



Event Subscribing Channel	Disabled (0) ▼
Output Policy	Bit will be Set ▼

参数说明：

Name: 用户自定义的引脚名称，设置为PIN_2。

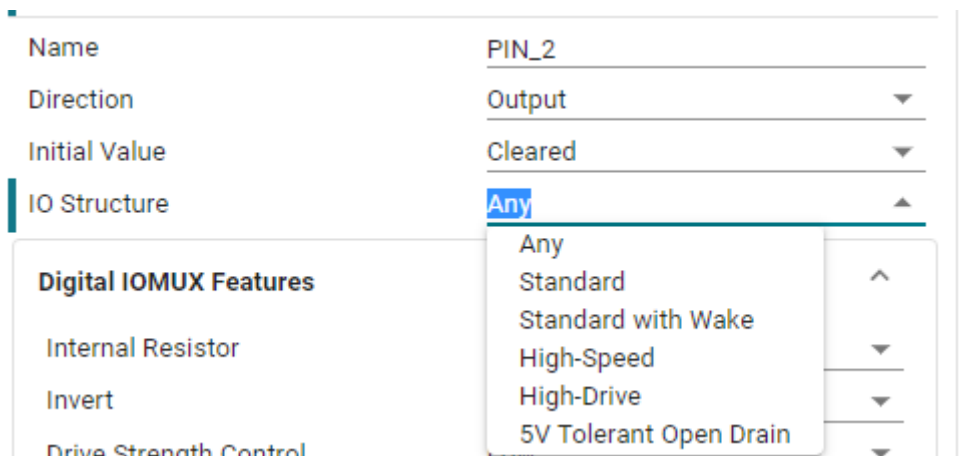
Direction: 设置引脚模式，输入和输出。这里我们是控制LED灯，选择为输出模式。

Name	PIN_2
Direction	Output ▼
Initial Value	Output ▼
IO Structure	Any ▼

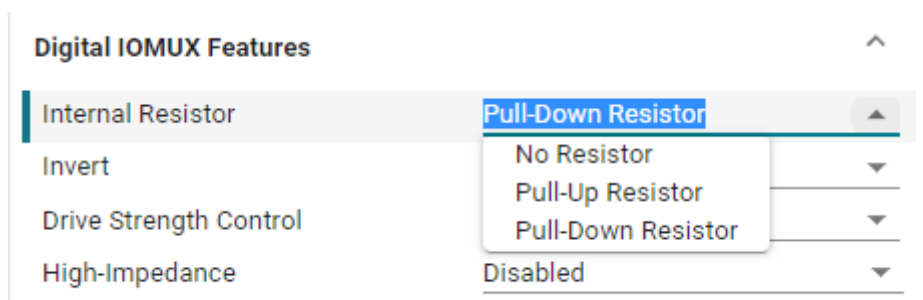
Initial Value: 设置引脚的初始状态，只有在配置为输出模式时才可以设置。有两个选项，清除和置位。清除即输出低电平，置位即输出高电平。

Name	PIN_2
Direction	Output ▼
Initial Value ?	Cleared ▼
IO Structure	Set ▼

IO Structure: 设置IO结构。有多个选项，默认(Any)、标准(Standard)、唤醒(Standard with Wake)、高速(High-Speed)以及可以容忍5V(5V Tolerant Open Drain)的结构。



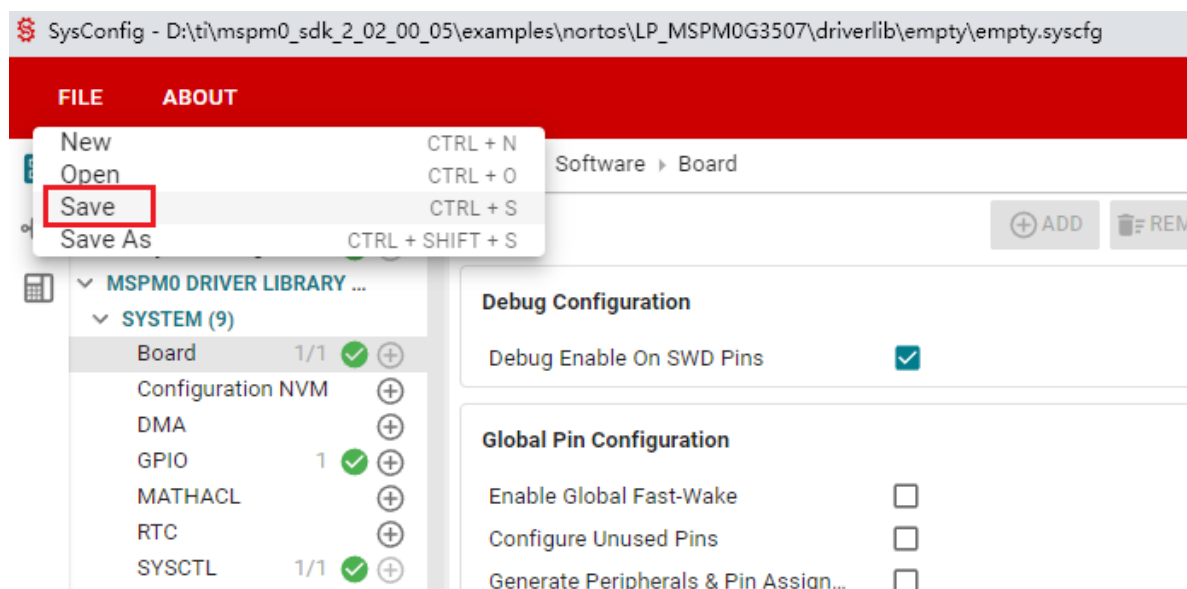
Internal Resistor: 设置引脚的上下拉电阻。有三个选项，不设置、设置上拉和设置下拉电阻。这里根据LED灯的接法，选择了下拉电阻。



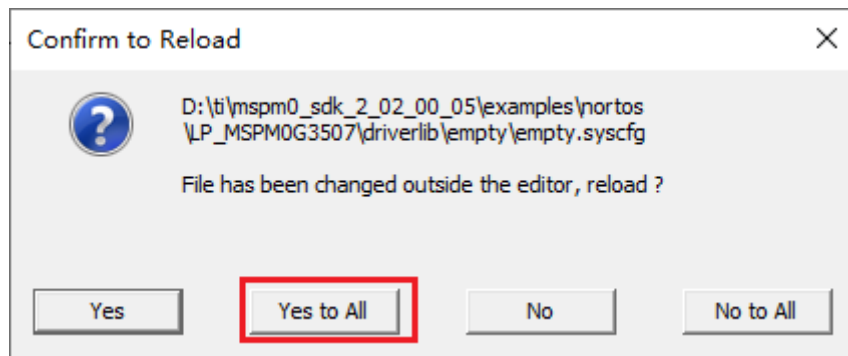
Assigned Pin: 设置引脚号。要控制哪一个引脚就填写对应的引脚号，比如开发板中的LED是接到了GPIOB2，那么就填入2。

2.3 保存更新配置文件

点击 SAVE 将 SYSCONFIG 中的配置保存，之后关掉 SYSCONFIG 回到keil。



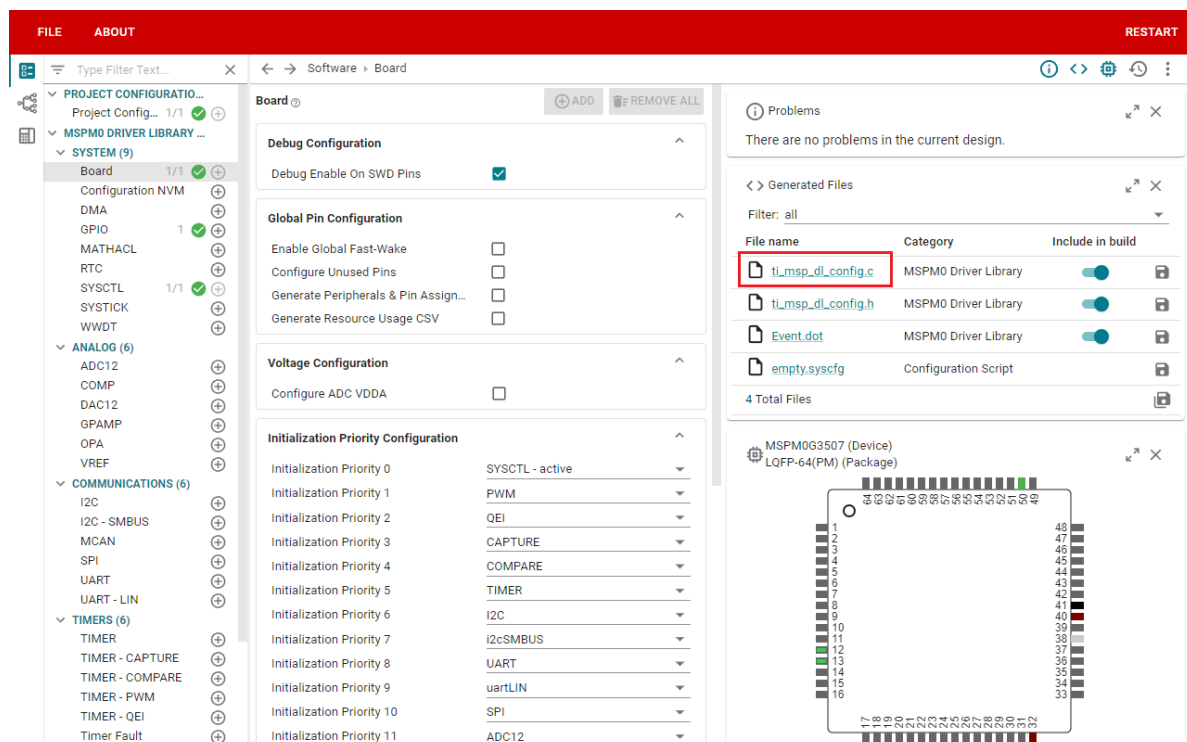
在弹窗中选择 Yes to All

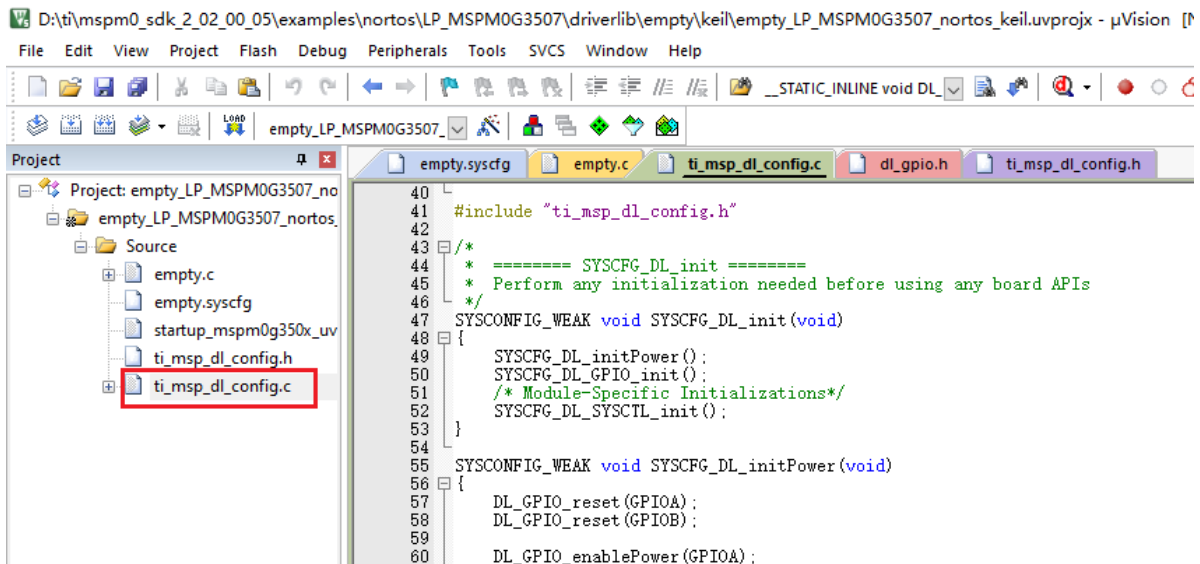
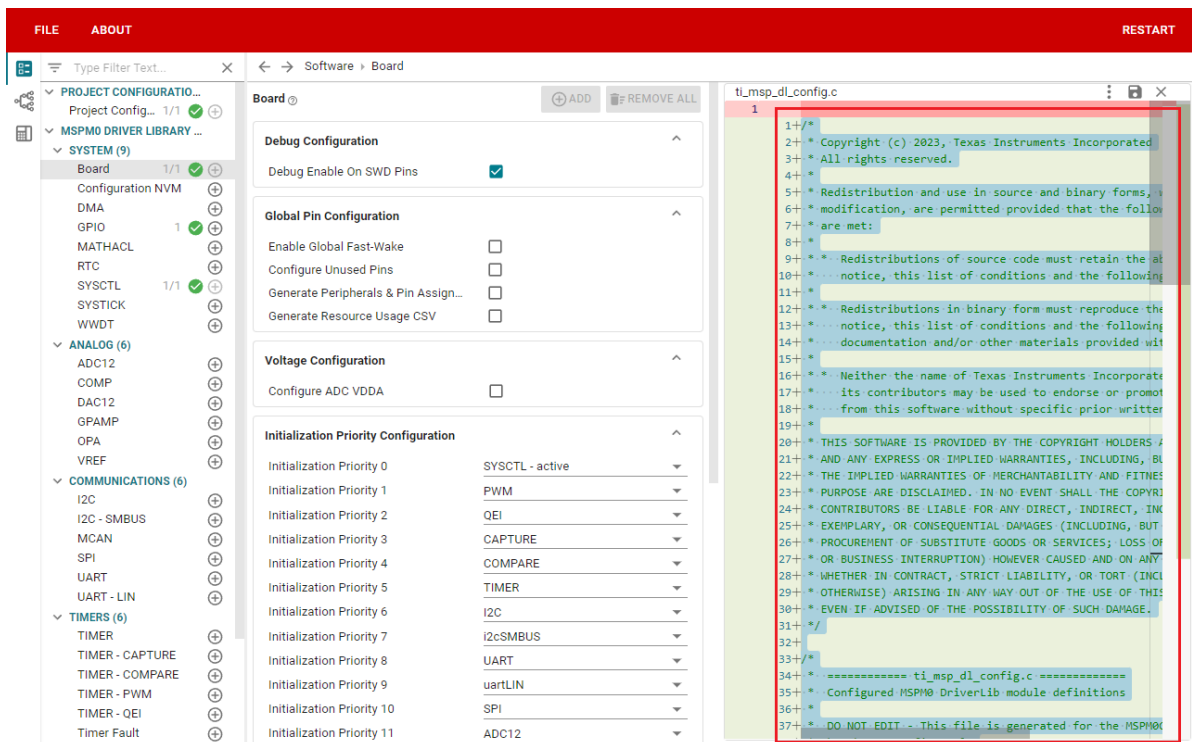


之后我们就可以看到empty.syscfg文件跟新成了我们设置的参数。

```
1 /**
2  * These arguments were used when this file was generated. They will be automatically applied on subsequent loads
3  * via the GUI or CLI. Run CLI with '--help' for additional information on how to override these arguments.
4  * @cliArgs --device "MSPM0G350X" --part "Default" --package "LQFP-64 (PM)" --product "mspm0_sdk@2.02.00.05"
5  * @v2CliArgs --device "MSPM0G3507" --package "LQFP-64 (PM)" --product "mspm0_sdk@2.02.00.05"
6  * @versions {"tool": "1.21.1+3772"}
7  */
8
9 /**
10  * Import the modules used in this configuration.
11  */
12 const GPIO = scripting.addModule("/ti/driverlib/GPIO", {}, false);
13 const GPIO1 = GPIO.addInstance();
14 const SYSCTL = scripting.addModule("/ti/driverlib/SYSCTL");
15
16 /**
17  * Write custom configuration values to the imported modules.
18  */
19 GPIO1.$name = "LED1";
20 GPIO1.port = "PORTB";
21 GPIO1.associatedPins[0].$name = "PIN_2";
22 GPIO1.associatedPins[0].internalResistor = "PULL_DOWN";
23 GPIO1.associatedPins[0].assignedPin = "2";
24
25 const Board = scripting.addModule("/ti/driverlib/Board", {}, false);
26
27 SYSCTL.forceDefaultClkConfig = true;
28
29 /**
30  * Pinnux solution for unlocked pins/peripherals. This ensures that minor changes to the automatic solver in a future
31  * version of the tool will not impact the pinnux you originally saw. These lines can be completely deleted in order to
32  * re-solve from scratch.
33  */
34 GPIO1.associatedPins[0].pin.$suggestSolution = "PB2";
35 Board.peripheral.$suggestSolution = "DEBUGSS";
36 Board.peripheral.swclkPin.$suggestSolution = "PA20";
37 Board.peripheral.swdioPin.$suggestSolution = "PA19";
38 SYSCTL.peripheral.$suggestSolution = "SYSCTL";
39
```

我们还需要确认.c和.h文件是否更新。直接编译，编译就会自动更新到keil中。如果没有更新，我们也可以把 SYSCONFIG 中的文件内容复制过来。





.h文件同上，可以将 SYSCONFIG 中的文件内容复制过来。

3. 写入程序

在empty.c文件中，写入以下代码

```
#include "ti_msp_dl_config.h"

//自定义延时（不精确） Custom delay (not precise)
void delay_ms(unsigned int ms)
{
    unsigned int i, j;
    // 下面的嵌套循环的次数是根据主控频率和编译器生成的指令周期大致计算出来的，
    // 需要通过实际测试调整来达到所需的延时。
    // The number of nested loops below is roughly calculated based on the master
    control frequency and the instruction cycle generated by the compiler, and needs
    to be adjusted through actual testing to achieve the required delay.
    for (i = 0; i < ms; i++)
    {
        for (j = 0; j < 8000; j++)
```



```

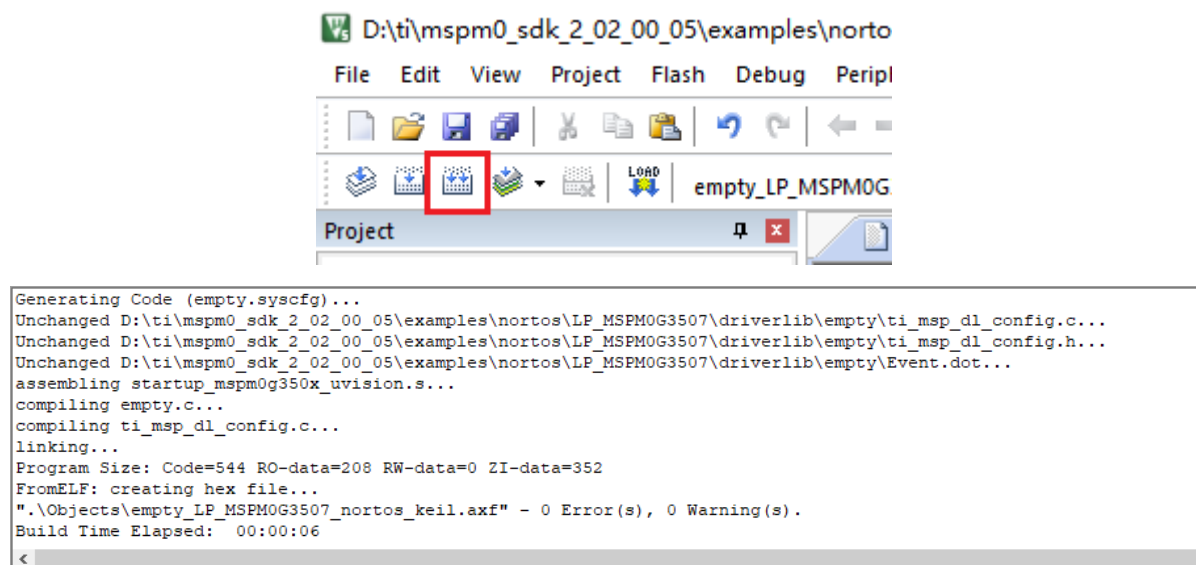
    {
        // 仅执行一个足够简单以致于可以预测其执行时间的操作
        // Perform only one operation that is simple enough to predict its
        execution time
        __asm__("nop"); // "nop" 代表“无操作”，在大多数架构中，这会消耗一个或几个时钟
        周期 "nop" stands for "no operation", which on most architectures consumes one or
        a few clock cycles
    }
}

int main(void)
{
    SYSCFG_DL_init();
    while (1)
    {
        DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN); //输出低电平 Output low level
        delay_ms(1000); //延时大概1S Delay about 1S
        DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN); //输出高电平 Output high level
        delay_ms(1000); //延时大概1S Delay about 1S
    }
}

```

4. 编译

点击 Rebuild 图标。出现以下提示表示编译完成，并且没有错误。



四、程序分析

- dl_gpio.h

```

/**
 * @brief      Set a group of GPIO pins
 *
 * @param[in]  gpio  Pointer to the register overlay for the peripheral
 * @param[in]  pins  Pins to set high. Bitwise OR of @ref DL_GPIO_PIN.
 */
__STATIC_INLINE void DL_GPIO_setPins(GPIO_Regs* gpio, uint32_t pins)
{
    gpio->DOUTSET31_0 = pins;
}

```

```

}

/**
 * @brief      Clear a group of GPIO pins
 *
 * @param[in]  gpio  Pointer to the register overlay for the peripheral
 * @param[in]  pins  Pins to clear. Bitwise OR of @ref DL_GPIO_PIN.
 */
__STATIC_INLINE void DL_GPIO_clearPins(GPIO_Regs* gpio, uint32_t pins)
{
    gpio->DOUTCLR31_0 = pins;
}

/**
 * @brief      Toggle a group of GPIO pins
 *
 * @param[in]  gpio  Pointer to the register overlay for the peripheral
 * @param[in]  pins  Pins to toggle. Bitwise OR of @ref DL_GPIO_PIN.
 */
__STATIC_INLINE void DL_GPIO_togglePins(GPIO_Regs* gpio, uint32_t pins)
{
    gpio->DOUTTGL31_0 = pins;
}

```

__STATIC_INLINE void DL_GPIO_setPins(GPIO_Regs* gpio, uint32_t pins): 该函数为控制引脚输出高电平。

__STATIC_INLINE void DL_GPIO_clearPins(GPIO_Regs* gpio, uint32_t pins): 该函数为控制引脚输出低电平。

__STATIC_INLINE void DL_GPIO_togglePins(GPIO_Regs* gpio, uint32_t pins): 该函数为控制引脚的电平翻转，如果原来是高电平则变为低电平，如果原来是低电平则变为高电平。

- empty.c

```

int main(void)
{
    SYSCFG_DL_init();
    while (1)
    {
        DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN); //输出低电平 Output low level
        delay_ms(1000); //延时大概1s Delay about 1s
        DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN); //输出高电平 Output high level
        delay_ms(1000); //延时大概1s Delay about 1s
    }
}

```

设置PB2输出低电平，延时大概1s后输出高电平，再延时大概1s。循环此进程。

五、实验现象

程序下载完成之后，可以看到MSPM0开发板上的LED1灯被点亮，并每隔大约1s闪烁一次。

